

Numpy Cheatsheet

1. Matrix Creation

```
```python
import numpy as np

np.zeros((3,3)) # 3x3 matrix of 0s
np.ones((2,4)) # 2x4 matrix of 1s
np.eye(5) # 5x5 identity matrix
np.arange(0,10,2) # [0 2 4 6 8]
np.linspace(0,1,5) # [0.0, 0.25, 0.5, 0.75, 1.0]
np.random.rand(3,3) # 3x3 uniform random [0,1)
np.random.randint(0,10,(2,3)) # 2x3 ints in [0,10)
```

## 2. Indexing & Slicing

```
arr = np.arange(25).reshape(5,5)

arr[2,3] # Element at row 2, col 3
arr[1:4, 0:3] # Rows 1-3, cols 0-2
arr[::2, :-1] # Every other row, reversed cols
arr[arr > 10] # Boolean indexing
arr[:, np.newaxis] # Add new axis (shape: 5x1x5)
```

## 3. Array Operations

```
Where/nonzero
np.where(arr > 5, arr, -1) # Replace values <5 with -1
indices = np.nonzero(arr) # Get indices of non-zero elements
bigger_than_7_indices = np.nonzero(arr > 7) # Get indices of elements >7

Unique elements
np.unique(arr, return_counts=True) # Values and counts

Array sizing
arr.shape # Tuple of dimension sizes (number of elements per dim)
arr.size # Total elements (prod(shape))
arr.ndim # Number of dimensions
```

## 4. Reshaping

```
arr.reshape(5, 5) # Reshape `arr` to 5x5. Returns view if possible
arr.flatten() # Flatten array. Always returns copy
arr.ravel() # Flatten array. Returns view if possible
np.expand_dims(arr, 0) # Add new dimension at axis 0
np.squeeze(arr) # Remove singleton dimensions (dim of size 1)
```

## 5. Padding

```
np.pad(arr, 1) # Default 0-padding
np.pad(arr, (1,2), mode='edge') # Replicate edge values
np.pad(arr, 2, mode='constant', constant_values=5) # Pad with 5s
```

## 6. Aggregation

```
arr.sum(axis=0) # Column sums
arr.mean(axis=1) # Row averages
np.min(arr, axis=(0,1)) # Min across both dimensions
np.max(arr, axis=1) # Max over rows (across columns: dim 1)
```

## 7. Iteration

```
1D
for val in arr.flat: print(val)

2D with indices
for idx, val in np.ndenumerate(arr):
 print(f"Index {idx}: {val}")

2D without indices
for row in arr:
 for val in row:
 print(val)
```

---

# Python Strings Cheatsheet

## Basic Operations

```
s = "Hello World"

s.upper() # "HELLO WORLD"
s.lower() # "hello world"
s.split() # ["Hello", "World"]
s.strip() # Remove whitespace
s.replace("World", "Python") # "Hello Python"
f"{s[:.5]}" # Truncate: "Hello"
```

## Type Checking

```
s.isalpha() # True if all alphabetic
s.isdigit() # True if all digits
s.isalnum() # Alpha-numeric check
```

```
s.startswith("Hello") # True

s.index("l") # index of first "l" (-1 if not found)
s.count("l") # number of "l"s
```

---

## Python Lists Cheatsheet

### Operations

```
lst = [1,2,3]

lst.index(2) # Returns 1
lst.append(4) # [1,2,3,4]
lst.extend([5,6]) # [1,2,3,4,5,6]
lst.insert(0, 0) # Insert at position 0
lst[::-1] # Reverse list
[x*2 for x in lst] # List comprehension
```

---

## Common Pitfalls

### Python Basics

#### 1. Mutable Default Arguments

```
def bad(a=[]): # Shared between calls!
def good(a=None): a = a or []
```

#### 2. Shallow Copying

```
a = [[1,2], [3,4]]
b = a.copy() # Outer list copied, inner lists referenced
use deep copy:
import copy
b = copy.deepcopy(a)
```

#### 3. Integer Division

```
3 / 2 # 1.5 in Python 3 (// for floor division)
```

## Numpy Gotchas

#### 1. View vs Copy

```
a = np.arange(10)
b = a[::2] # View (modifying b affects a)
c = a[::2].copy() # Independent copy
```

## 2. Boolean Indexing

```
Wrong: arr = arr[arr > 5] and arr[arr < 3]
Correct: mask = (arr > 5) | (arr < 3); arr = arr[mask]
```

## 3. Broadcasting Errors

```
a = np.ones((3,4))
b = np.ones((4,3))
a + b # Error! Use a + b.T
```

## 4. In-Place Operations

```
a = np.array([1,2,3], dtype=np.int8)
a += 100 # Overflow! (Becomes [101, 102, 103] % 256)
```

## 5. Axis Confusion

```
arr.sum(axis=0) # Sum columns (axis=0=rows dimension)
arr.sum(axis=1) # Sum rows (axis=1=columns dimension)
```

## 6. Mixed Data Types

```
np.array([1, "2"]) # dtype='<U11' (all elements become strings)
```

## 7. Empty Slices

```
arr = np.array([])
arr.mean() # Returns nan! Check size first
```

## 8. Integer Index Casting

```
arr[1.0] # Error! Index must be integer
```

## 9. Order of Operations

```
a = np.array([1,2,3])
b = a * 2 + 1 # Equivalent to (a * 2) + 1
```